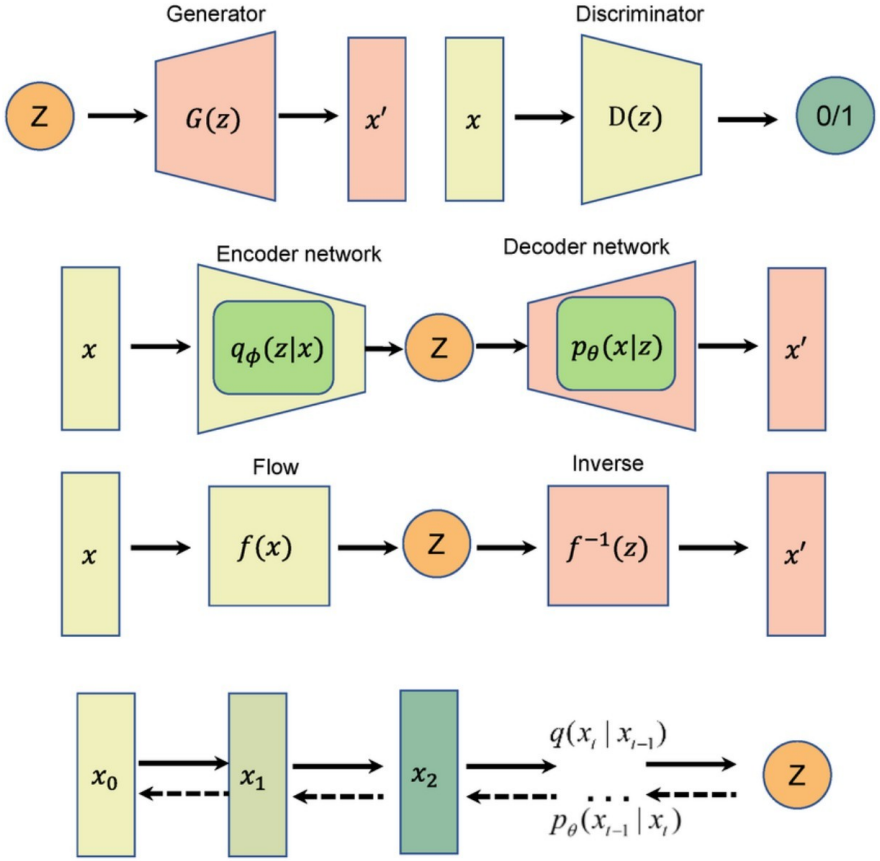
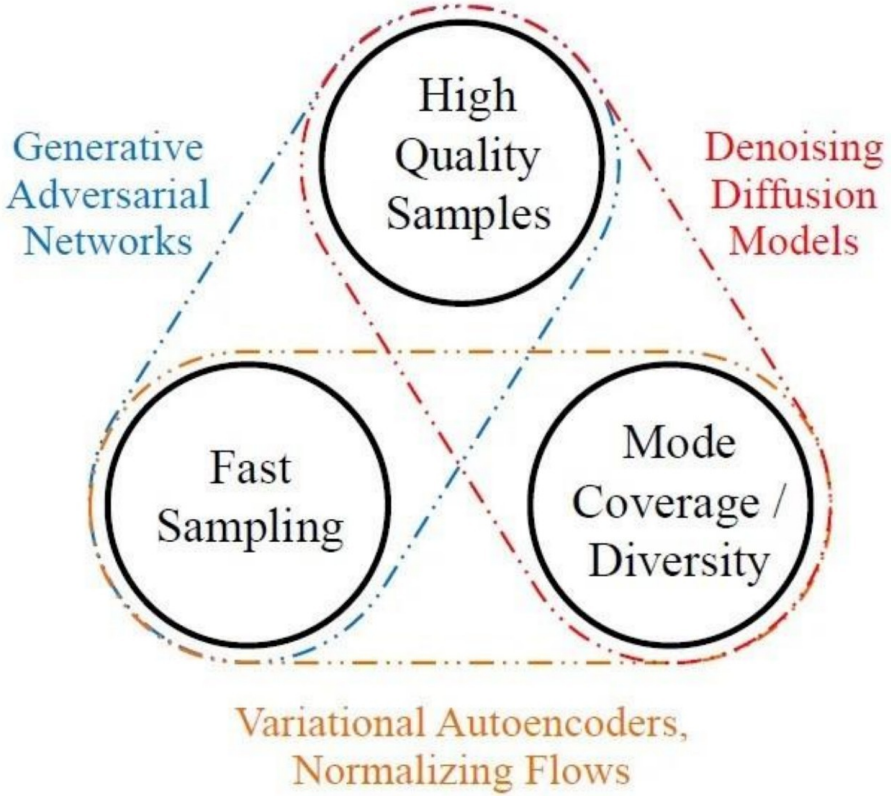
Генеративные модели Computer Vision

Угадай семейство моделей

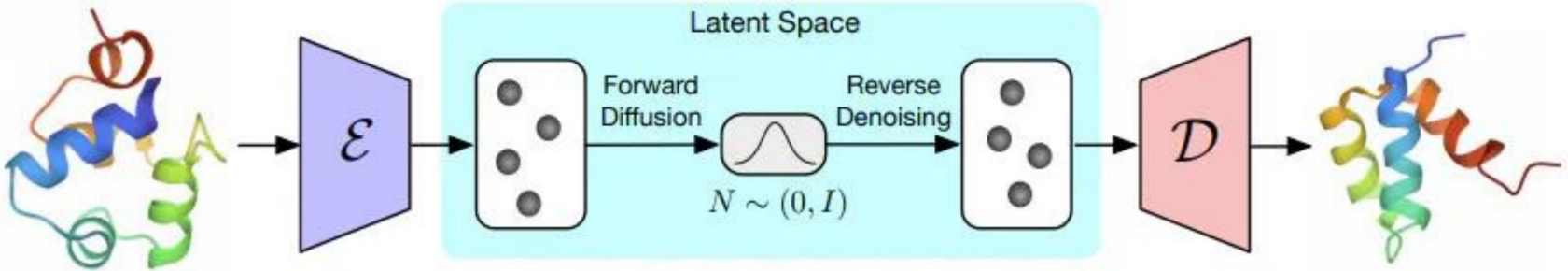


Generative learning trilemma



Source: https://zhuanlan.zhihu.com/p/503932823

Stable Diffusion (Diffusion + VAE + CLIP)



Добавление CLIP-эмбеддингов

Cross-attention:

- У нас есть внутренние представления картинки на текущем шаге диффузии (features)
- У нас есть CLIP-эмбеддинг текста (тоже в виде векторов)

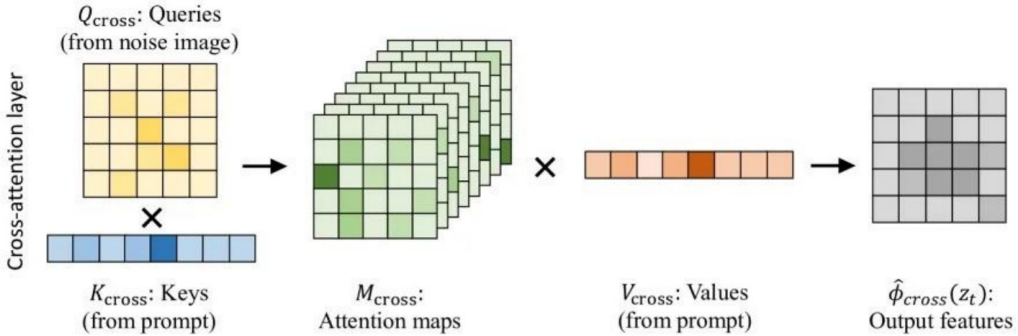
UNet делает cross-attention:

- Для каждого «пиксельного» или patch-представления картинки
- Смотрит на все векторы из текстового эмбеддинга
- Решает, какая часть текста важна для этого участка изображения

Модель взвешивает информацию из текста и картинки, чтобы обновить шум так, чтобы он стал ближе к описанию.

Добавление CLIP-эмбеддингов

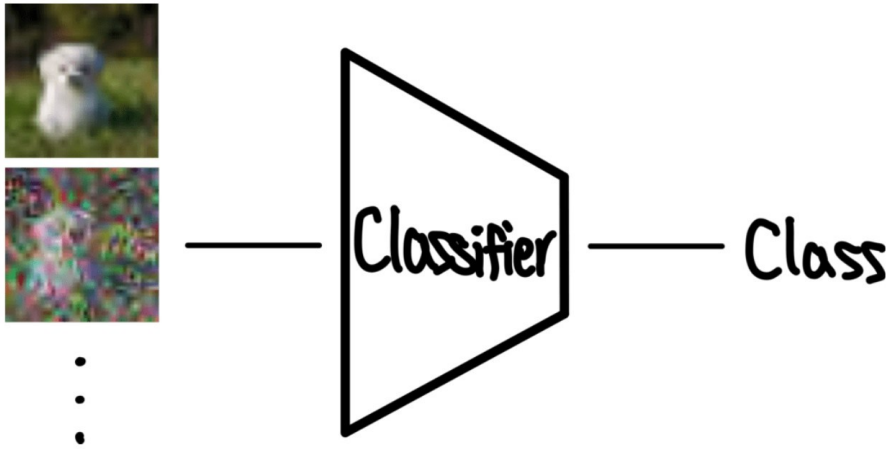
1. **Queries (Q)** — это патчи картинки (текущие feature-векторы в UNet на шаге диффузии).
   - Каждая маленькая часть картинки “спрашивает”: «На что из текста мне обратить внимание?»
2. **Keys (K)** и **Values (V)** — это векторы слов из CLIP-эмбеддинга текста
   - Каждое слово текста превращается в вектор.
   - Keys отвечают за «идентификацию важности слова», Values — за информацию, которую можно использовать.
3. **Attention**:
   1. Для каждого патча Query считается вес сходства с каждым Key → softmax → веса внимания
   2. Query суммирует Values, взвешенные этими коэффициентами → обновляет патч картинки



Classifier quidance

Часто генерация по промпту бывает неуверенной или нечеткой.

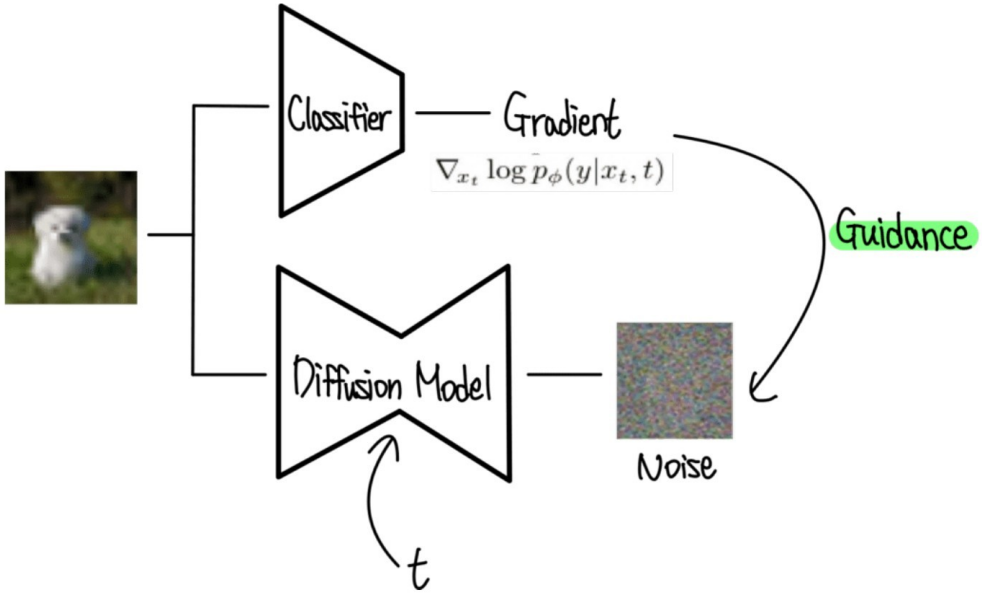
Можно дополнительно обучить классификатор, который будет определять уверенность модели в сгенерированном классе и на каждой итерации генерации сдвигать результат в сторону нужного класса.



Classifier quidance: применение

Часто генерация по промпту бывает неуверенной или нечеткой.

Можно дополнительно обучить классификатор, который будет определять уверенность модели в сгенерированном классе и на каждой итерации генерации сдвигать результат в сторону нужного класса.



Classifier-free guidance (CFG)

На этапе обучения модели с некоторой небольшой фиксированной вероятностью мы подаем пустой промпт (вместо изначального промпта)

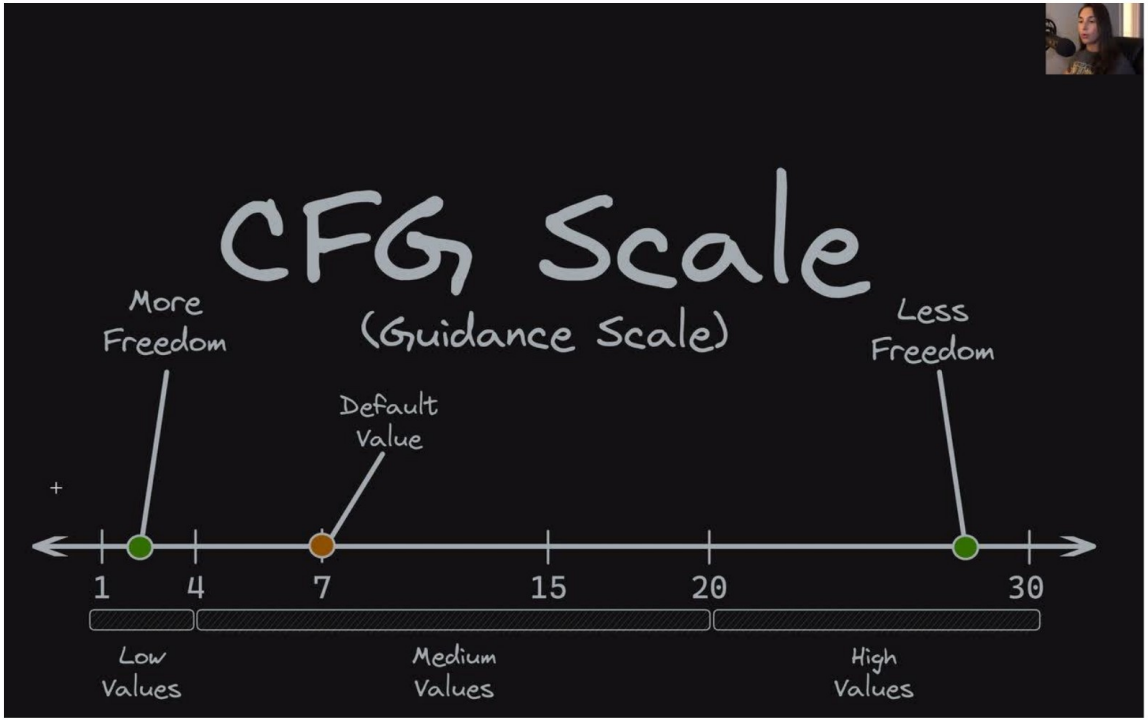
В диффузионных моделях **CFG** позволяет управлять генерацией изображения по тексту **без отдельного классификатора**, используя один и тот же шумовой предсказатель:

$$\hat{\epsilon} = \epsilon_\theta(x_t) + w \cdot (\epsilon_\theta(x_t, c) - \epsilon_\theta(x_t))$$

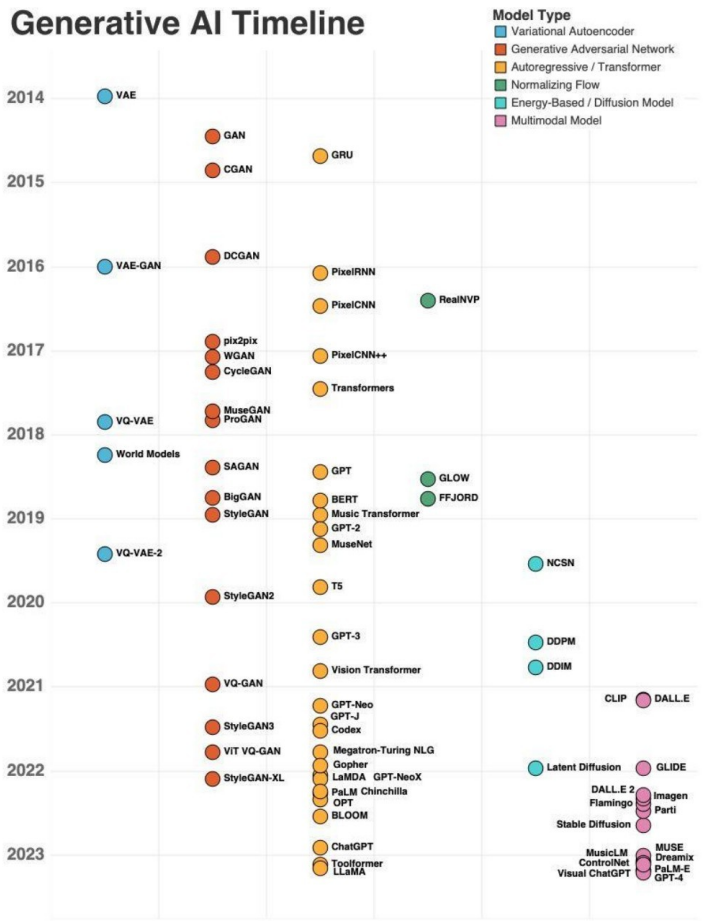
- $\epsilon_\theta(x_t, c)$ — предсказание модели с условием c (текст).
- $\epsilon_\theta(x_t)$ — предсказание модели **без условия** (пустой промпт).
- w — коэффициент “guidance scale”, который усиливает влияние текста.

Classifier-free guidance (CFG)

На этапе обучения модели с некоторой небольшой фиксированной вероятностью мы подаем пустой промпт (вместо изначального промпта)



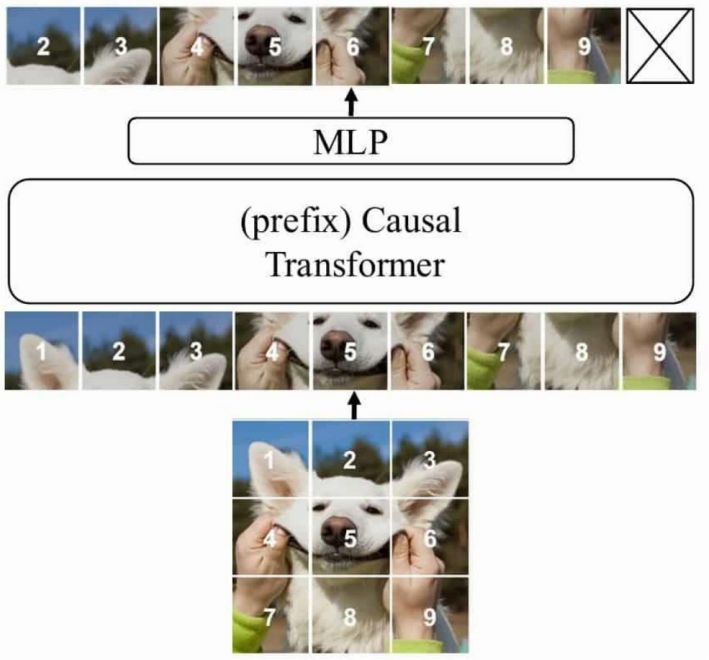
Timeline генеративных CV-моделей



Авторегрессионные модели

Идея AR-моделей

Авторегрессионные модели (как и в NLP) учатся прогнозировать кусочки изображения “токен за токеном”

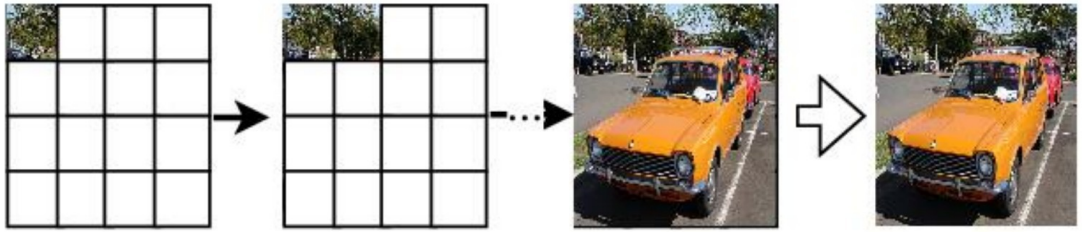


Идея AR-моделей

Токен картинки (или же patch) – это кусочек изображения, одна “клеточка”.

- Изображение разбивается на патчи, затем эти патчи векторизуются (для каждого патча модель выучит числовой вектор-embedding) и подаются в модель

Next-Patch Prediction [Taming Transformers, 2020]



Сравнение AR-моделей с диффузией

Генерация по классу, ImageNet 256×256; все модели замерены в одинаковых условиях (одна A100, 1 картинка)

Модель	Как генерирует	FID ↓ качество	Precision ↑ реалистичность	Recall ↑ разнообразие	Шагов	Время на 1 картинку
DiT-XL/2 (0.7B)	диффузия	2.27	0.83	0.57	250	4.3 с
LlamaGen-XXL (1.4B)	AR: токен за токеном	2.34	0.80	0.59	576	24.4 с
RAR-XXL (1.5B)	AR: токен за токеном	1.48	0.80	0.63	256	6.6 с
VAR-d30 (2.0B)	AR: от грубого к точному (по масштабам)	1.92	0.82	0.59	10	0.26 с

- **Качество:** современные AR-модели не хуже диффузии — RAR и VAR обходят DiT по FID.
- **Разнообразие и реалистичность:** примерно на одном уровне (Recall 0.57–0.63, Precision 0.80–0.83).
- **Скорость решает способ генерации:** AR «токен за токеном» (сотни шагов) медленнее диффузии, а VAR, предсказывающая сразу целый масштаб за 10 шагов, — примерно в 17 раз быстрее DiT.

Источник: сводная таблица из «Locality-aware Parallel Decoding for Efficient Autoregressive Image Generation» (arXiv 2507.01957). FID — общее качество, Precision — доля реалистичных картинок, Recall — насколько полно покрыто разнообразие данных.

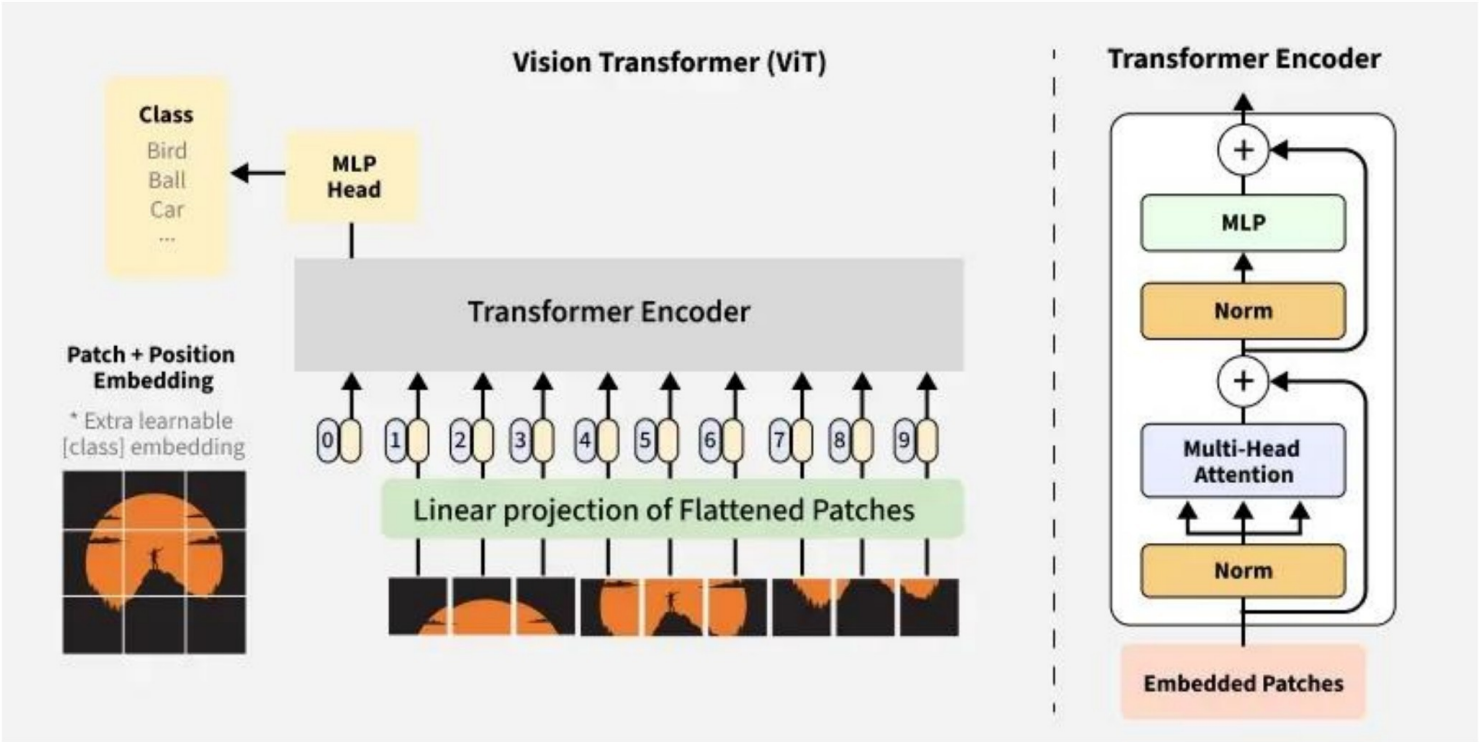
Типы архитектур

- **PixelAR / PixelCNN / PixelRNN**
  - Генерируют изображение пиксель за пикселем
  - Медленно генерируют и редко используются сейчас

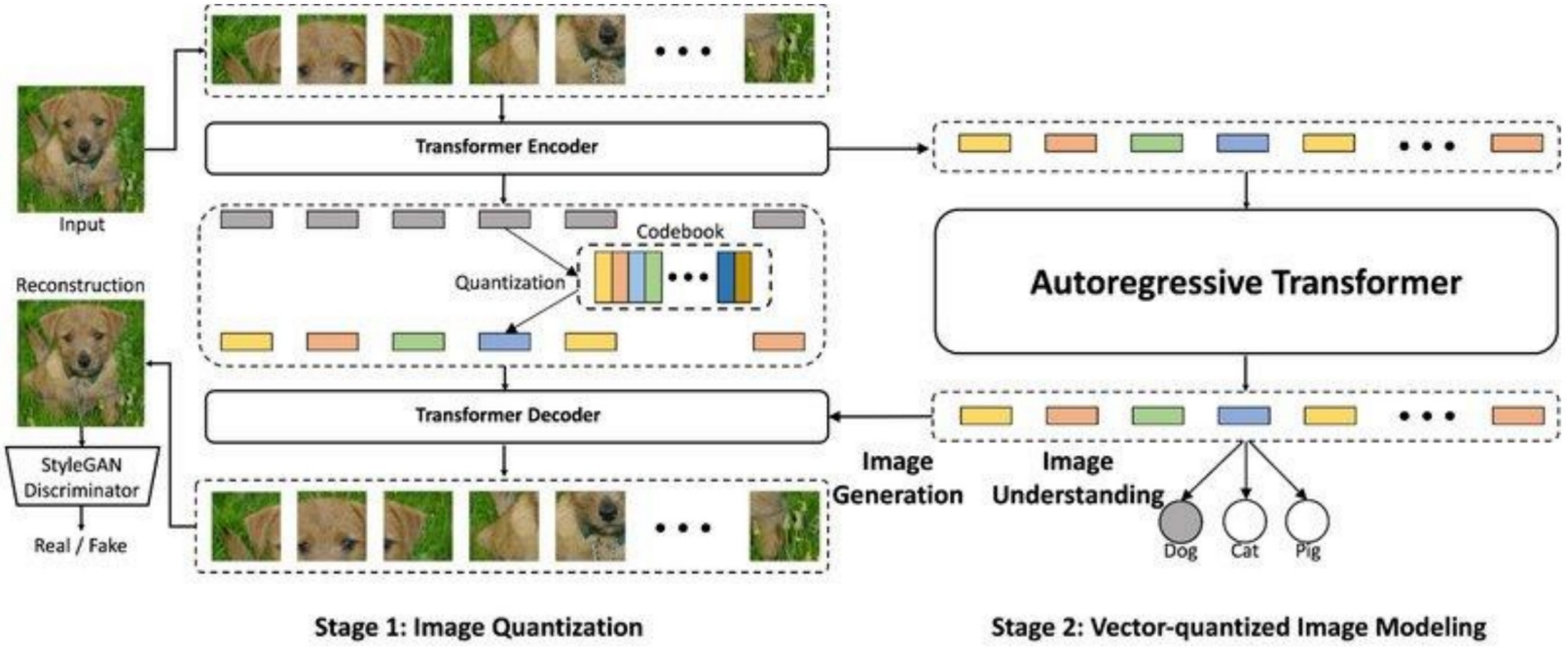
- **VQ-AR (через токены)**
  - Изображение разбивают на токены
  - Потом Transformer генерирует эти токены один за другим
  Пример: DALL·E 1, RQ-Transformer

- **Иерархический подход**
  - Сначала модель генерирует крупные “размытые” блоки
  - Потом в несколько шагов уточняет их содержимое

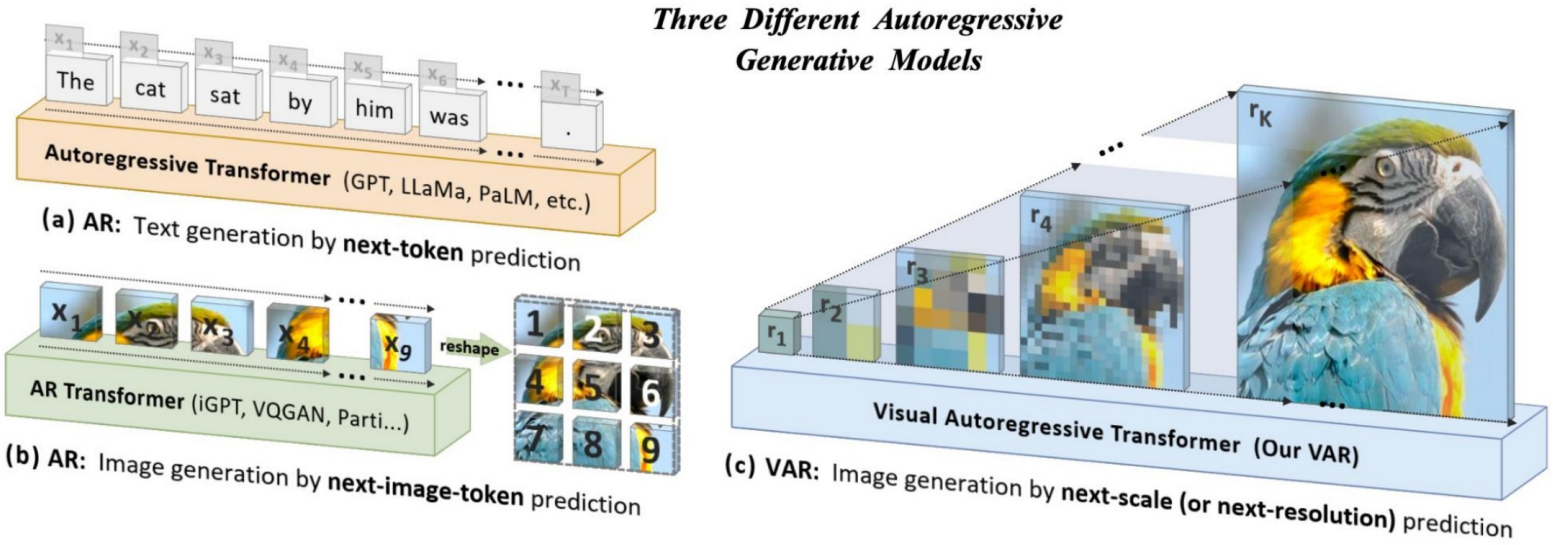
Vision Transformer (аналог BERT)



Vector Quantized AR-model, VQ-AR (аналог GPT)



Coarse-to-fine иерархический подход



Текстовая диффузия

Чем нас не устраивают авторегрессионные модели?

Минусы AR-моделей

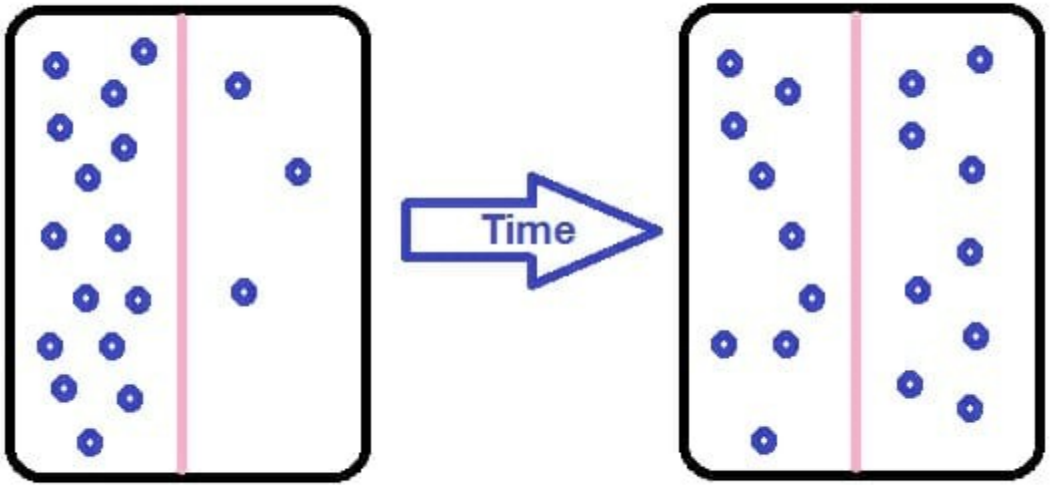
1) Не может исправлять ошибки текста после генерации символа
2) Дорогой и долгий
3) Не естественно для правок (ссылки, точечные исправления и так далее)

Пример



[Why does ChatGPT sometimes contradict itself within the same sentence? : r/ChatGPT]

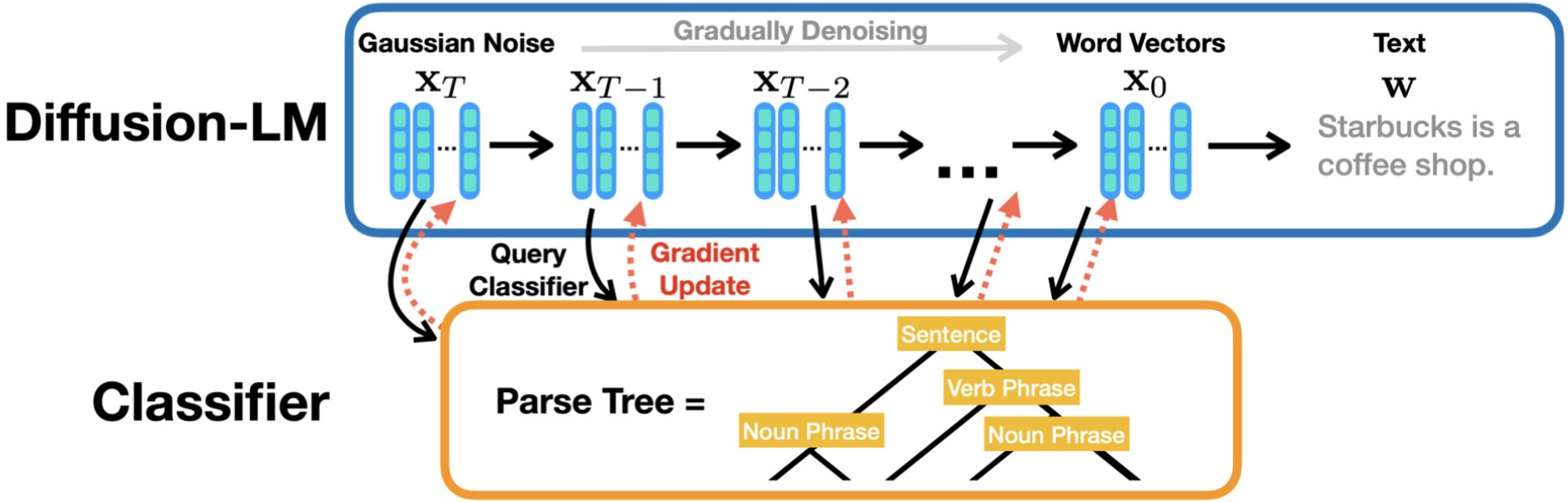
Решение! Давайте делать диффузию



Diffusion - Definition, Examples and Types | Biology Dictionary

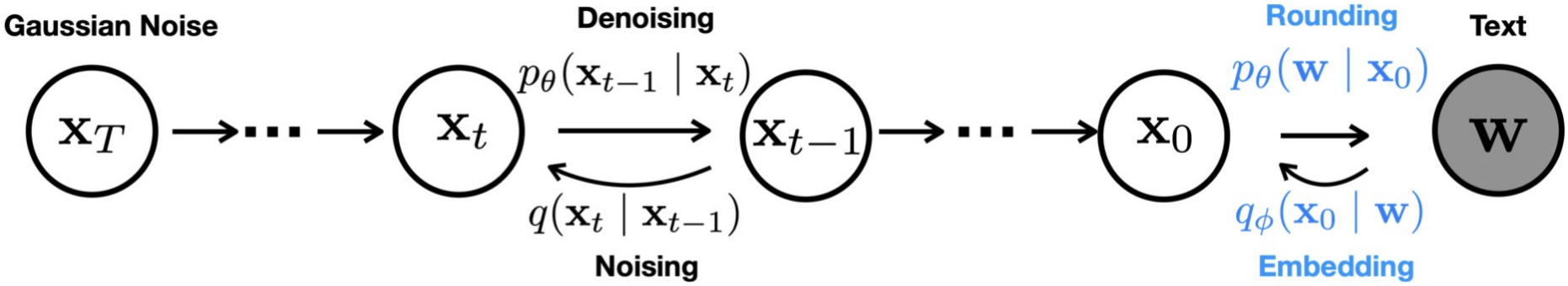
https://arxiv.org/pdf/2205.14217

А как делать? Diffusion LM



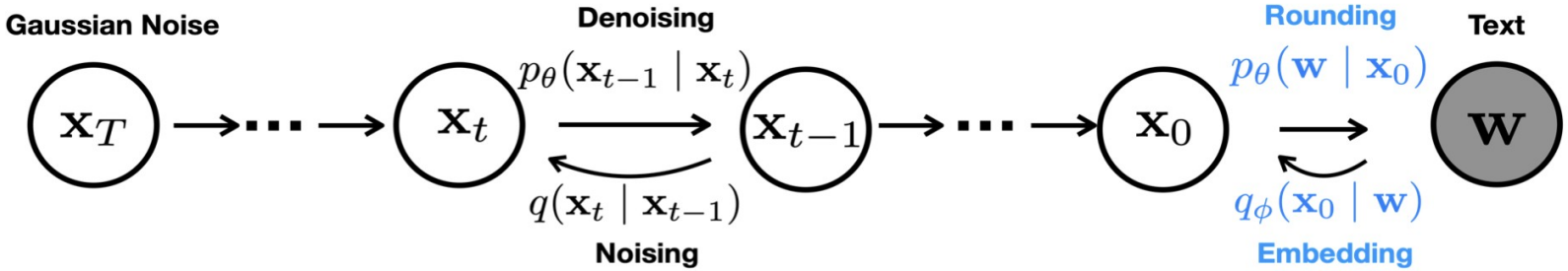
https://arxiv.org/pdf/2205.14217

Диффузионная языковая модель



https://arxiv.org/pdf/2205.14217

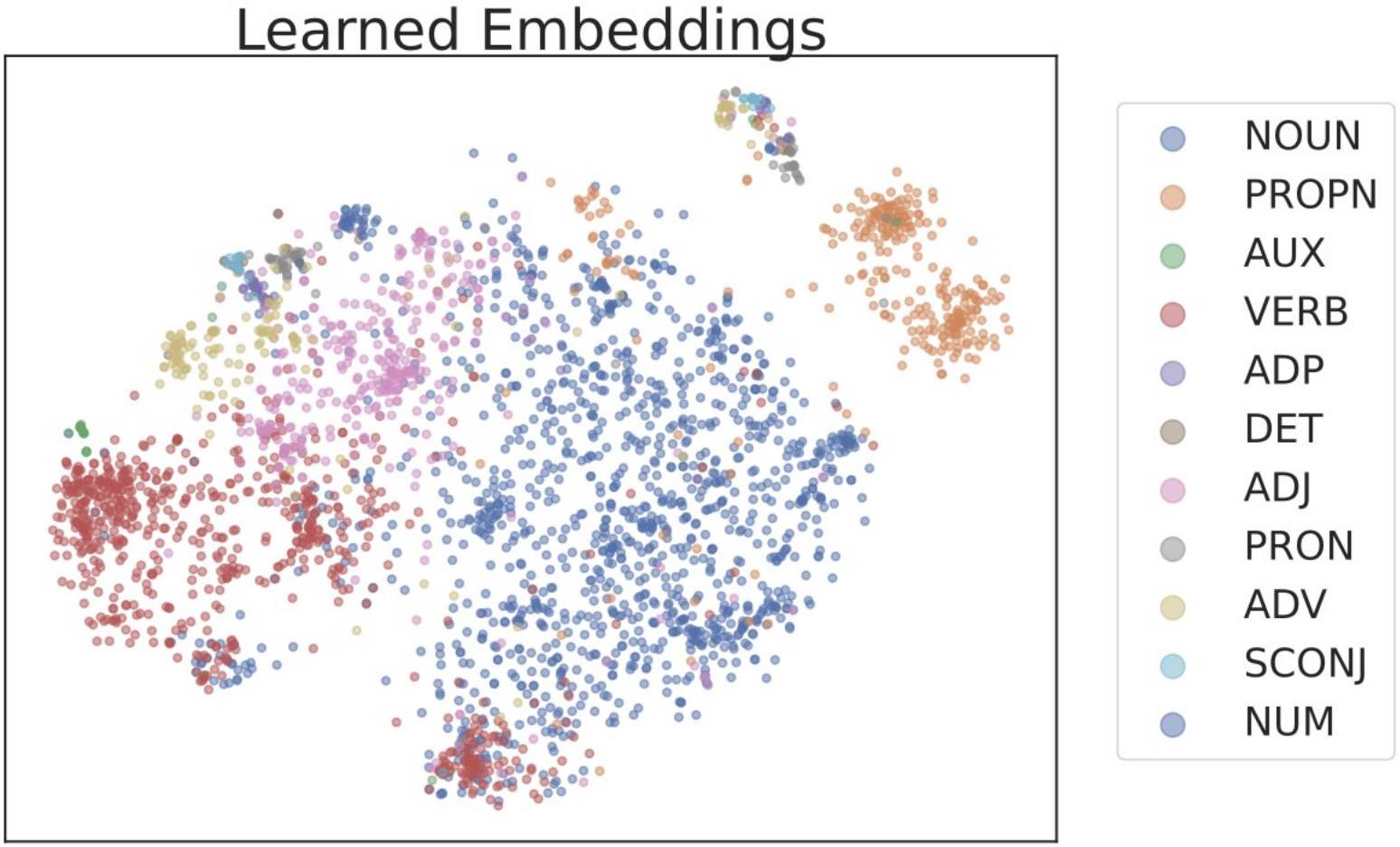
Диффузионная языковая модель



$$\mathcal{L}(\mathbf{w}) = \mathbb{E}\Big[\underbrace{\sum_{t=1}^{T} \big\| f_\theta(\mathbf{x}_t, t) - \mathbf{x}_0 \big\|^2}_{\text{(1) диффузионное}} + \underbrace{\big\| \mathrm{EMB}(\mathbf{w}) - \mathbf{x}_0 \big\|^2}_{\text{(2) эмбеддинговое}} \underbrace{- \log p_\theta(\mathbf{w} \mid \mathbf{x}_0)}_{\text{(3) rounding}} \Big]$$

$$\mathbf{x}_0 \sim \mathcal{N}\big(\mathrm{EMB}(\mathbf{w}), \sigma_0^2 I\big), \qquad p_\theta(\mathbf{w} \mid \mathbf{x}_0) = \prod_{i=1}^{n} \mathrm{softmax}\big(W x_{0,i}\big)_{w_i}$$

https://arxiv.org/pdf/2205.14217



https://arxiv.org/pdf/2205.14217

Проблемы округления

The learned embeddings define a mapping from discrete text to the continuous $\mathbf{x}_0$. We now describe the inverse process of rounding a predicted $\mathbf{x}_0$ back to discrete text. Rounding is achieved by choosing the most probable word for each position, according to argmax $p_\theta(\mathbf{w} \mid \mathbf{x}_0) = \prod_{i=1}^{n} p_\theta(w_i \mid x_i)$. Ideally, this argmax-rounding would be sufficient to map back to discrete text, as the denoising steps should ensure that $\mathbf{x}_0$ lies exactly on the embedding of some word. However, empirically, the model fails to generate $\mathbf{x}_0$ that commits to a single word.

https://arxiv.org/pdf/2212.09462

Stable diffusion для текста

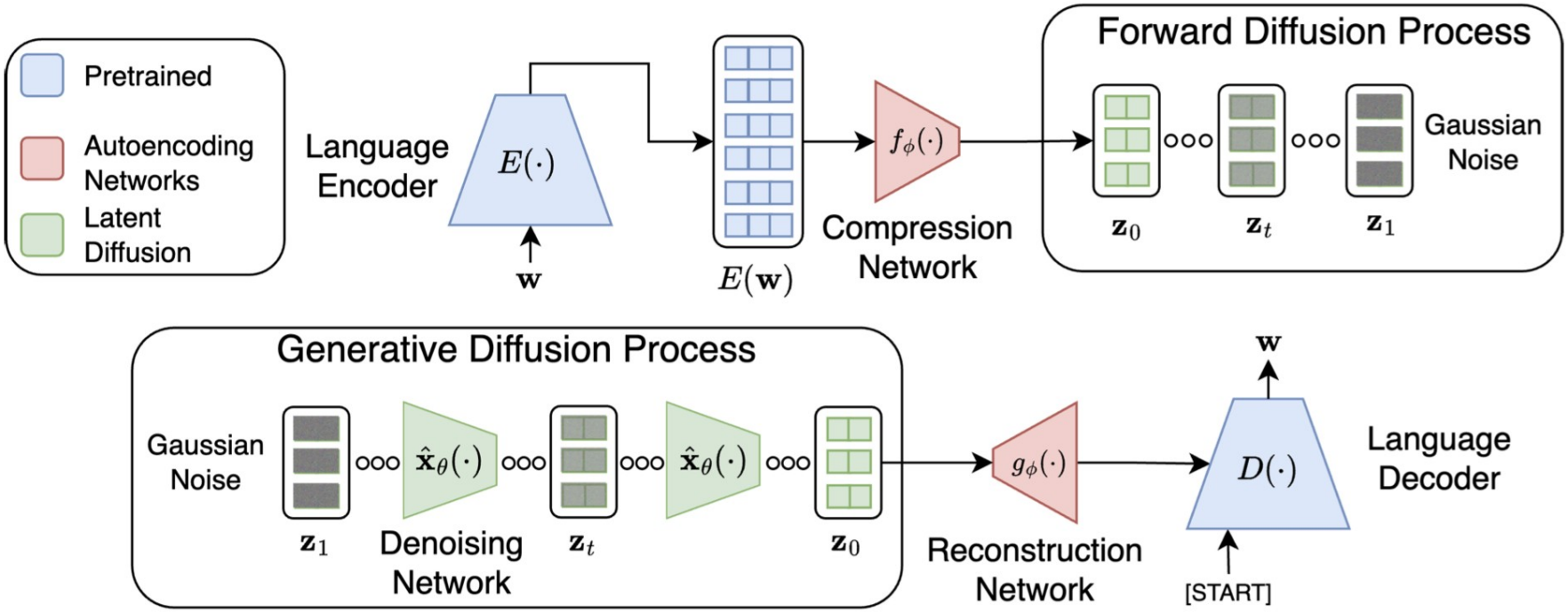


Figure 1: Overview of our proposed latent language diffusion framework.

https://arxiv.org/pdf/2212.09462

Как оценивают качество?

We use **MAUVE Score** [47] and **Perplexity** (Ppl) to evaluate the quality of our generated text. MAUVE Score is a metric for open-ended text generation that compares the distribution of generated text with that of reference text using divergence frontiers. We follow Pillutla et al. [47] and use the GPT-2-Large model [49] to embed the text. Perplexity measures how likely the generated samples are according to an autoregressive language model; we use GPT-2-Large to compute perplexity.

We also want to quantify the **Diversity** (Div) of generations. We define diversity as $\textbf{Div} = \prod_{n=2}^{4} \frac{|\text{unique n-grams}(\{\mathbf{w}_i\})|}{|\text{total n-grams}(\{\mathbf{w}_i\})|}$ where $\{\mathbf{w}_i\}$ is a set of generated samples [68]. The metrics discussed so far can be optimized by generating samples from the training set. We measure the proportion of generated 4-grams that are found in the training set to quantify the degree of **Memorization** (Mem).

https://arxiv.org/pdf/2212.09462

Сравнение моделей

		ROCStories				AG News			
	Timesteps	MAUVE ↑	Ppl ↓	Div ↑	Mem ↓	MAUVE ↑	Ppl ↓	Div ↑	Mem ↓
Reference	-	$.951_{.007}$	$21.1_{.3}$	$.414_{.003}$	$.362_{.003}$	$.951_{.014}$	$43.6_{1.2}$	$.658_{.002}$	$.385_{.005}$
Diffusion-LM [36]	2000	$.043_{.006}$	$47.3_{.6}$	$.128_{.002}$	$.434_{.002}$	$.012_{.001}$	$67.1_{1.2}$	$.043_{.002}$	$.086_{.006}$
LD4LG (BART-Base)	250	$.716_{.019}$	$30.6_{.5}$	$.331_{.005}$	$.441_{.004}$	$.866_{.016}$	$100.6_{2.9}$	$.540_{.006}$	$.293_{.001}$
LD4LG (FLAN-T5-base)	250	$.481_{.007}$	$37.5_{.4}$	$.389_{.002}$	$.387_{.002}$	$.859_{.020}$	$122.0_{3.9}$	$.624_{.008}$	$.221_{.003}$
GPT-2-Medium	-	$.788_{.025}$	$20.0_{.2}$	$.372_{.002}$	$.688_{.006}$	$.820_{.012}$	$37.3_{1.1}$	$.532_{.017}$	$.829_{.005}$

https://arxiv.org/pdf/2212.09462

Еще сравнение

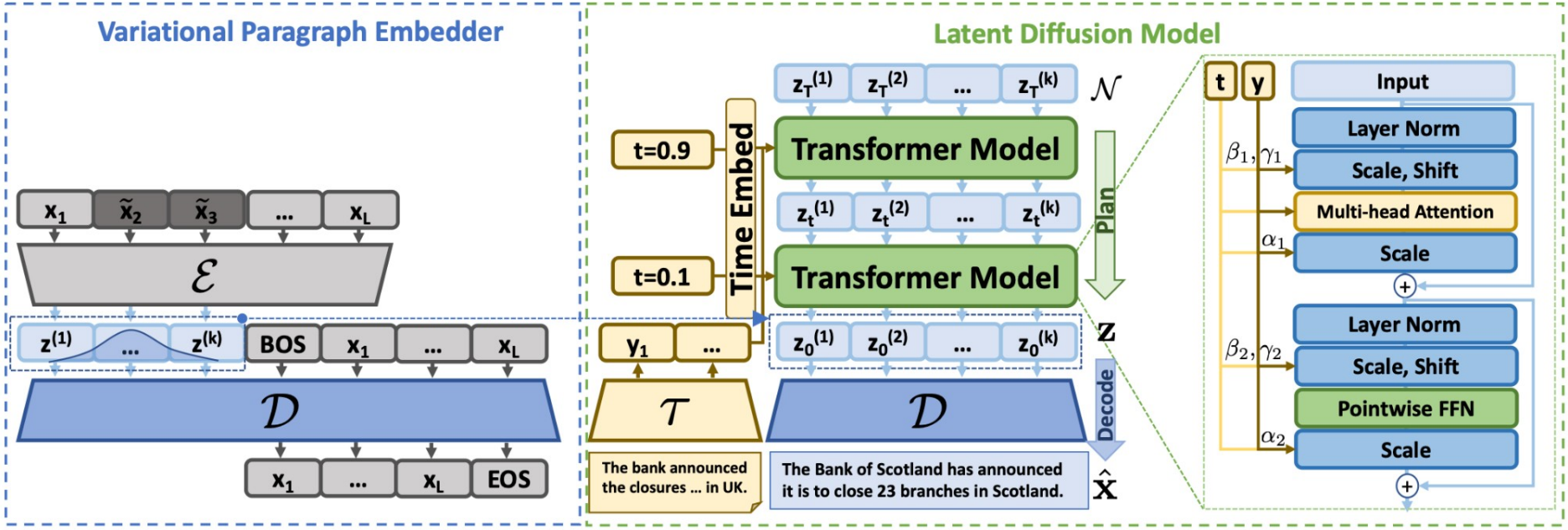
Table 5: Metrics for class-conditional generation.

	Conditioning	LD4LG (BART-base) MAUVE ↑ World	Sports	Business	Sci/Tech	Mem ↓	LD4LG (FLAN-T5-base) MAUVE ↑ World	Sports	Business	Sci/Tech	Mem ↓
Diffusion	World	$.842_{.017}$	$.015_{.002}$	$.026_{.002}$	$.020_{.002}$	$.296_{.002}$	$.809_{.024}$	$.013_{.001}$	$.025_{.002}$	$.022_{.002}$	$.233_{.005}$
	Sports	$.013_{.001}$	$.845_{.024}$	$.011_{.001}$	$.010_{.000}$	$.305_{.003}$	$.011_{.001}$	$.836_{.020}$	$.009_{.000}$	$.009_{.000}$	$.249_{.004}$
	Business	$.024_{.002}$	$.011_{.001}$	$.752_{.030}$	$.068_{.005}$	$.363_{.009}$	$.025_{.003}$	$.011_{.001}$	$.765_{.016}$	$.076_{.008}$	$.244_{.004}$
	Sci/Tech	$.023_{.002}$	$.012_{.001}$	$.082_{.008}$	$.813_{.028}$	$.225_{.004}$	$.024_{.001}$	$.011_{.001}$	$.082_{.010}$	$.843_{.033}$	$.169_{.004}$

	Conditioning	Conditional GPT-2 MAUVE ↑ World	Sports	Business	Sci/Tech	Mem ↓	Reference MAUVE ↑ World	Sports	Business	Sci/Tech	Mem ↓
Comparisons	World	$.805_{.022}$	$.012_{.000}$	$.025_{.002}$	$.021_{.002}$	$.402_{.002}$	$.963_{.009}$	$.018_{.001}$	$.034_{.002}$	$.032_{.003}$	$.388_{.007}$
	Sports	$.017_{.001}$	$.840_{.019}$	$.012_{.001}$	$.013_{.001}$	$.369_{.004}$	$.018_{.001}$	$.958_{.007}$	$.014_{.001}$	$.014_{.002}$	$.346_{.002}$
	Business	$.037_{.003}$	$.012_{.001}$	$.629_{.029}$	$.069_{.007}$	$.479_{.007}$	$.040_{.005}$	$.014_{.001}$	$.968_{.009}$	$.125_{.009}$	$.441_{.003}$
	Sci/Tech	$.033_{.002}$	$.013_{.001}$	$.102_{.015}$	$.697_{.027}$	$.434_{.004}$	$.036_{.003}$	$.016_{.001}$	$.133_{.013}$	$.955_{.011}$	$.366_{.003}$

https://arxiv.org/pdf/2306.02531

Более сложные архитектуры



https://arxiv.org/pdf/2306.02531

Люди думают Apple не занимается нейронками

Yizhe Zhang, Jiatao Gu, Zhuofeng Wu, Shuangfei Zhai, Josh Susskind, Navdeep Jaitly
Apple
{yizzhang, jgu32, zhuofeng_wu, szhai, jsusskind, njaitly}@apple.com

https://arxiv.org/pdf/2502.09992

Сравнение с AR-моделями

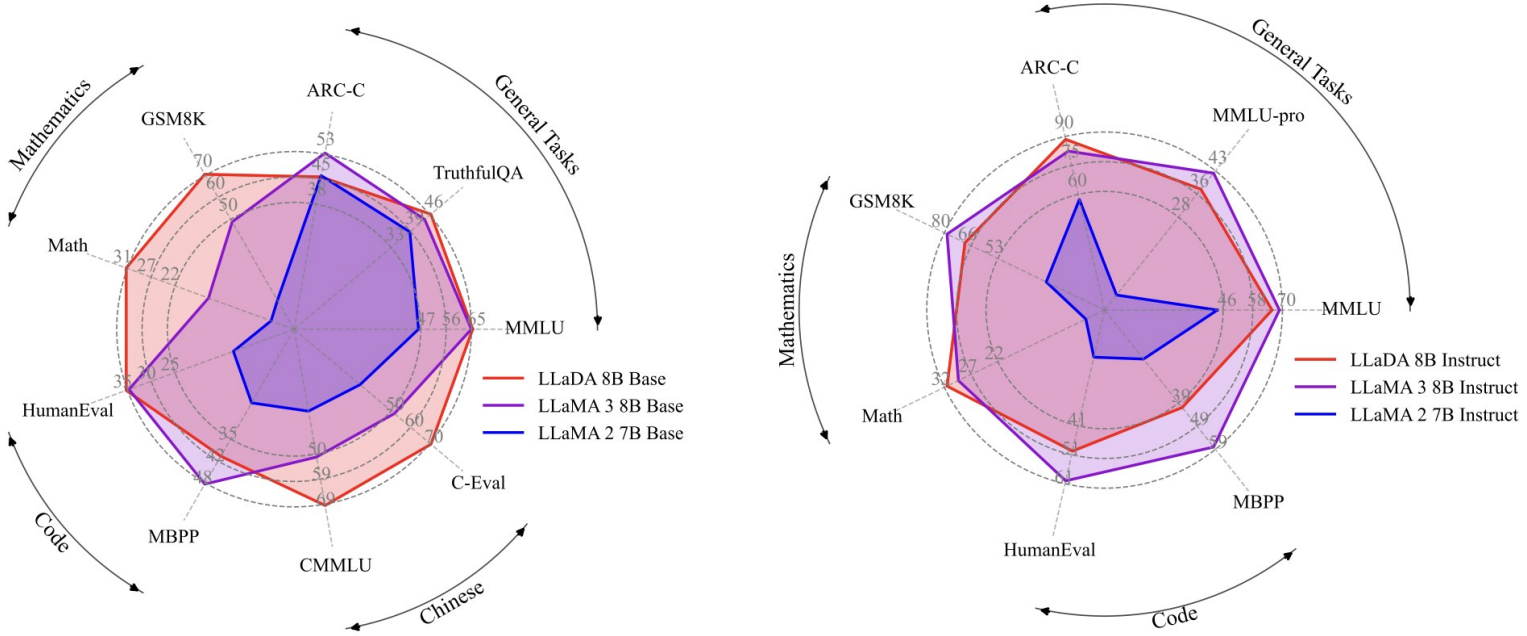


Figure 1: **Zero/Few-Shot Benchmarks.** We scale LLaDA to 8B parameters from scratch and observe competitive zero/few-shot performance compared with strong autoregressive LLMs [6].

https://arxiv.org/pdf/2502.09992

Архитектура

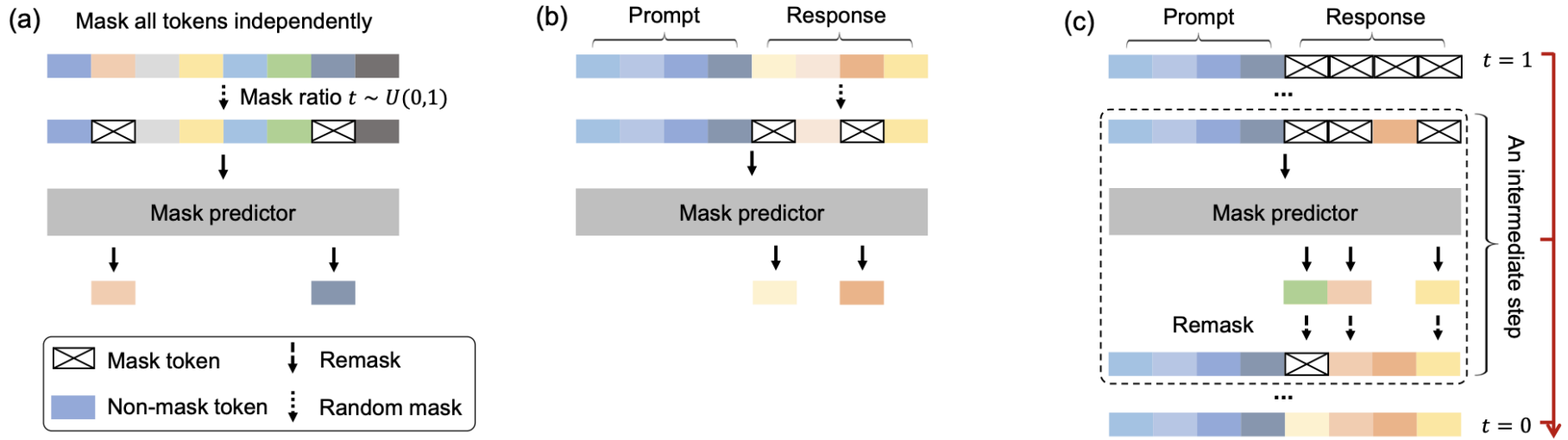
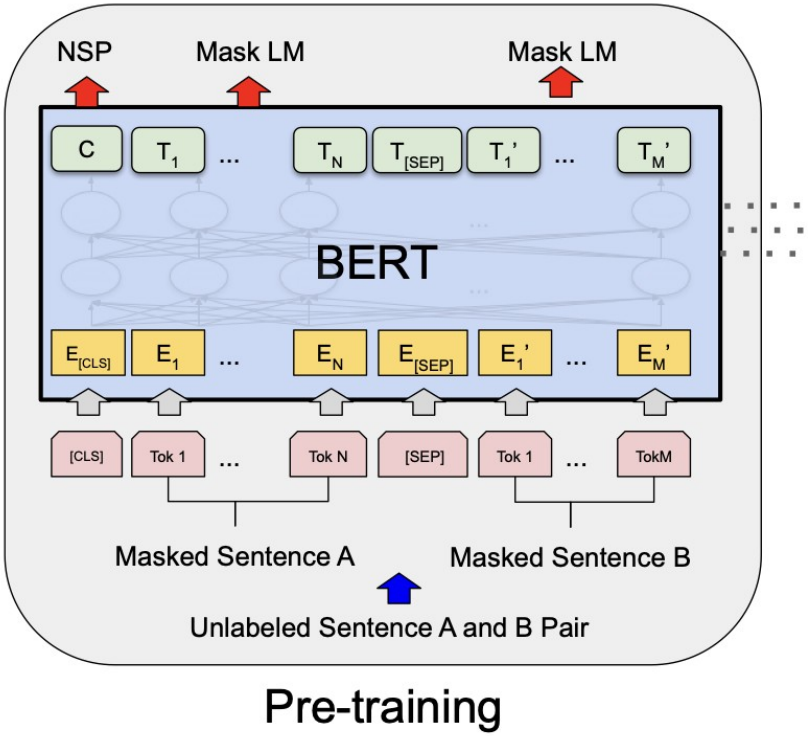


Figure 2: **Overview of LLaDA.** (a) Pre-training. LLaDA is trained on text with random masks applied independently to all tokens at the same ratio $t \sim U[0, 1]$. (b) SFT. Only response tokens are possibly masked. (c) Sampling. LLaDA simulates a diffusion process from $t = 1$ (fully masked) to $t = 0$ (unmasked), predicting all masks simultaneously at each step with flexible remask strategies.

https://arxiv.org/pdf/1810.04805

Похоже на BERT



https://arxiv.org/pdf/1810.04805

tokens. We refer to this procedure as a "masked LM" (MLM), although it is often referred to as a *Cloze* task in the literature (Taylor, 1953). In this case, the final hidden vectors corresponding to the mask tokens are fed into an output softmax over the vocabulary, as in a standard LM. In all of our experiments, we mask 15% of all WordPiece tokens in each sequence at random. In contrast to denoising auto-encoders (Vincent et al., 2008), we only predict the masked words rather than reconstructing the entire input.

Although this allows us to obtain a bidirectional pre-trained model, a downside is that we are creating a mismatch between pre-training and fine-tuning, since the `[MASK]` token does not appear during fine-tuning. To mitigate this, we do not always replace "masked" words with the actual `[MASK]` token. The training data generator chooses 15% of the token positions at random for prediction. If the i-th token is chosen, we replace

https://arxiv.org/pdf/2502.09992

Результаты

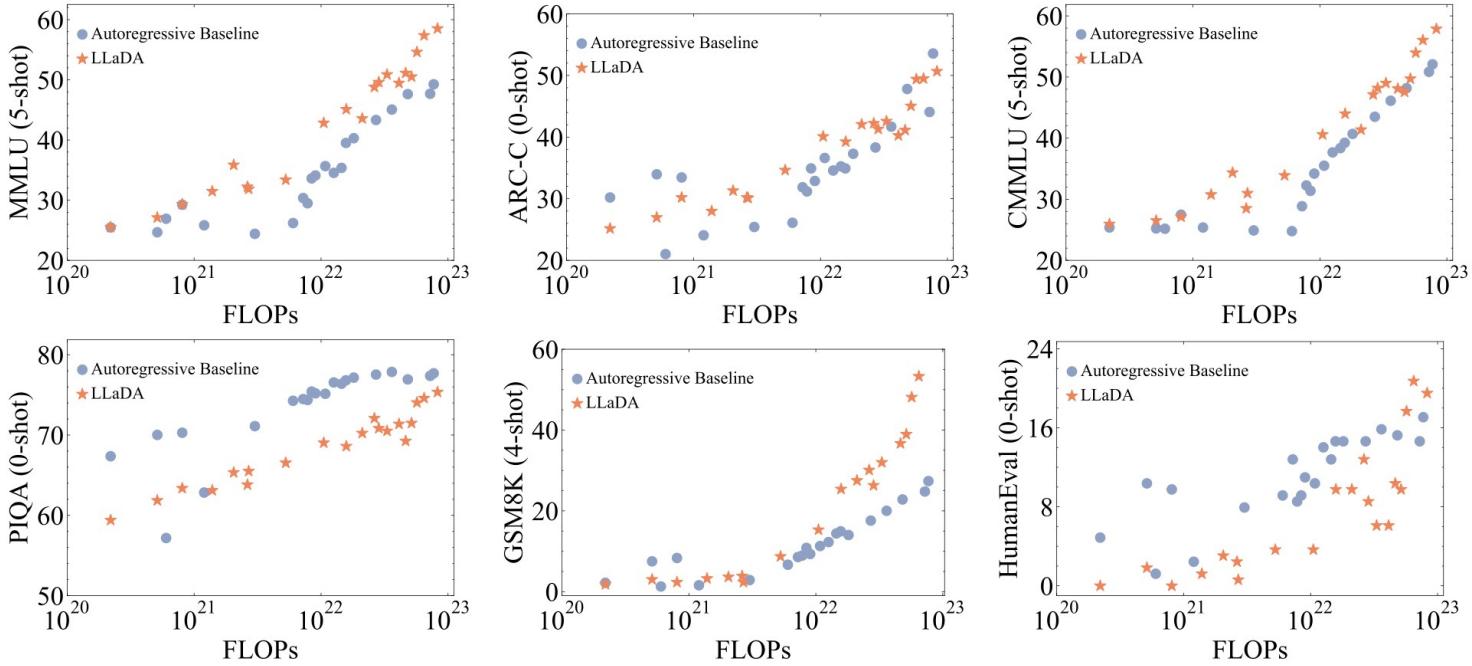


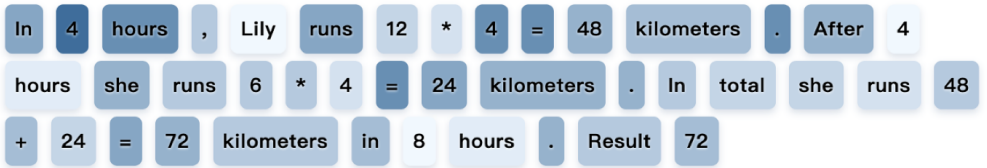
Figure 3: **Scalability of LLaDA.** We evaluate the performance of LLaDA and our ARM baselines trained on the same data across increasing pre-training computational FLOPs. LLaDA exhibits strong scalability, matching the overall performance of ARMs on six tasks.

https://arxiv.org/pdf/2502.09992

Еще метрики

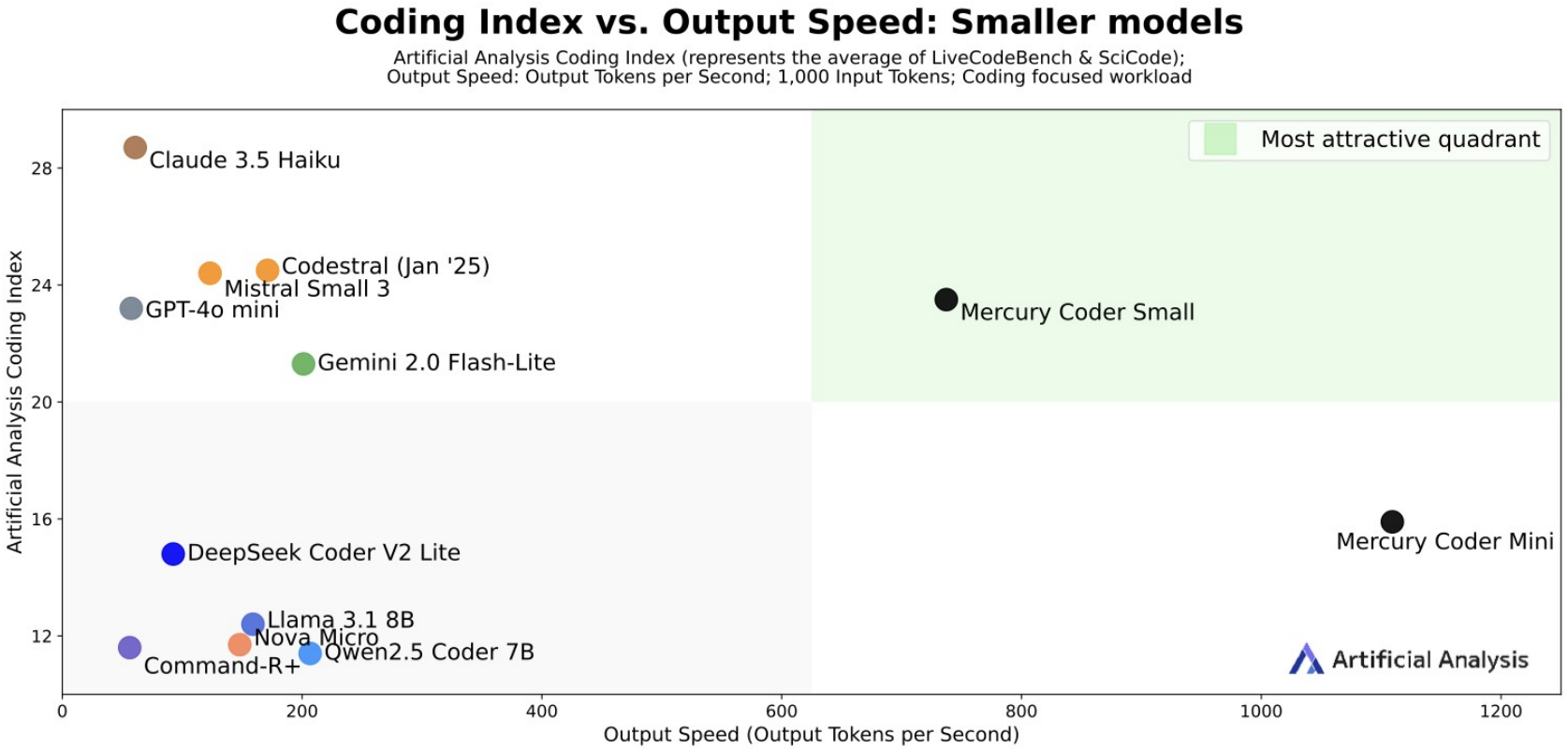
	LLaDA 8B*	LLaMA3 8B*	LLaMA2 7B*	Qwen2 7B†	Qwen2.5 7B†	Mistral 7B†	Deepseek 7B¶
Model	Diffusion	AR	AR	AR	AR	AR	AR
Training tokens	2.3T	15T	2T	7T	18T	-	2T
General Tasks							
MMLU	**65.9** (5)	65.4 (5)	45.9 (5)	70.3 (5)	74.2 (5)	64.2 (5)	48.2 (5)
BBH	49.7 (3)	**62.1** (3)	39.4 (3)	62.3 (3)	70.4 (3)	56.1 (3)	39.5 (3)
ARC-C	45.9 (0)	**53.1** (0)	46.3 (0)	60.6 (25)	63.7 (25)	60.0 (25)	48.1 (0)
Hellaswag	70.5 (0)	**79.1** (0)	76.0 (0)	80.7 (10)	80.2 (10)	83.3 (10)	75.4 (0)
TruthfulQA	**46.1** (0)	44.0 (0)	39.0 (0)	54.2 (0)	56.4 (0)	42.2 (0)	-
WinoGrande	74.8 (5)	**77.3** (5)	72.5 (5)	77.0 (5)	75.9 (5)	78.4 (5)	70.5 (0)
PIQA	73.6 (0)	**80.6** (0)	79.1 (0)	-	-	-	79.2 (0)
Mathematics & Science							
GSM8K	**70.3** (4)	48.7 (4)	13.1 (4)	80.2 (4)	85.4 (4)	36.2 (4)	17.4 (8)
Math	**31.4** (4)	16.0 (4)	4.3 (4)	43.5 (4)	49.8 (4)	10.2 (4)	6.0 (4)
GPQA	25.2 (5)	**25.9** (5)	25.7 (5)	30.8 (5)	36.4 (5)	24.7 (5)	-
Code							
HumanEval	**35.4** (0)	34.8 (0)	12.8 (0)	51.2 (0)	57.9 (0)	29.3 (0)	26.2 (0)
HumanEval-FIM	**73.8** (2)	73.3 (2)	26.9 (2)	-	-	-	-
MBPP	40.0 (4)	**48.8** (4)	23.2 (4)	64.2 (0)	74.9 (0)	51.1 (0)	39.0 (3)
Chinese							
CMMLU	**69.9** (5)	50.7 (5)	32.5 (5)	83.9 (5)	-	-	47.2 (5)
C-Eval	**70.5** (5)	51.7 (5)	34.0 (5)	83.2 (5)	-	-	45.0 (5)

https://arxiv.org/pdf/2502.09992

Sampling Process	
User	Lily can run 12 kilometers per hour for 4 hours. After that, she runs 6 kilometers per hour. How many kilometers can she run in 8 hours?
LLaDA	
Multi-round Dialogue Case	
User	Could you please share with me what the initial two lines of the renowned poem 'The Road Not Taken'?
LLaDA	Certainly! The initial two lines of the renowned poem "The Road Not Taken" by Robert Frost are: "Two roads diverged in a yellow wood, And sorry I could not travel both." These lines set the stage for the poem, introducing the idea of a speaker facing a choice between two paths.
User	Please help me translate into Chinese.
LLaDA	Sure! The first two lines of "The Road Not Taken" by Robert Frost can be translated into Chinese as: "两条路分岔在黄色的树林中，遗憾我不能同时走"
User	Please translate into German.
LLaDA	Sure! The first two lines of "The Road Not Taken" by Robert Frost can be translated into German as: "Zwei Wege trennten sich im gelben Wald, und leider konnte ich nicht beide tun."
User	Please also write a poem about life choices. I require 5 sentences, and the first word of each sentence starts with C as the first letter.
LLaDA	Certainly! Here's a poem about life choices, with each sentence starting with the letter "C": Choices cascade through life's journey, Creating connections with each decision, Careful consideration leads to better paths, Courageous choices shape our destiny, Choosing wisely guides us through the years.

https://arxiv.org/pdf/2506.17298

Mercury



https://arxiv.org/pdf/2506.17298

Model	HumanEval*	MBPP	EvalPlus	MultiPL-E	LCB*	BCB	Speed
Open-Weights Models							
Llama 3.1 8B Instruct	66.5	59.2	60.2	50.1	12.0	32.3	153
DeepSeek Coder V2 Lite	79.0	59.8	68.3	57.0	16.0	44.4	93
Mistral Small 3	84.8	69.6	72.3	70.1	25.0	42.7	126
Qwen 2.5 Coder 7B Instruct	88.0	80.0	79.3	75.3	9.0	41.4	195
Frontier Speed-Optimized Models							
Nova Micro	79.3	65.4	72.1	56.7	14.0	-	148
Codestral 2501	85.0	72.2	75.6	73.4	24.0	46.1	171
GPT 4o Mini	88.0	74.6	78.5	72.0	23.0	46.8	59
Claude 3.5 Haiku	86.0	78.0	75.1	72.3	31.0	45.4	61
Gemini 2.0 Flash Lite	90.0	75.0	77.3	79.5	18.0	44.4	201
Frontier Models							
DeepSeek V3	92.1	81.0	82.1	79.1	36.0	50.0	27
Claude 3.5 Sonnet	90.2	81.2	77.3	81.9	38.0	44.8	76
GPT 4o	90.2	82.2	82.4	77.6	31.0	49.9	61
Our Models							
Mercury Coder Mini	88.0	77.1	78.6	74.1	17.0	42.0	1109
Mercury Coder Small	90.0	76.6	80.4	76.2	25.0	45.5	737

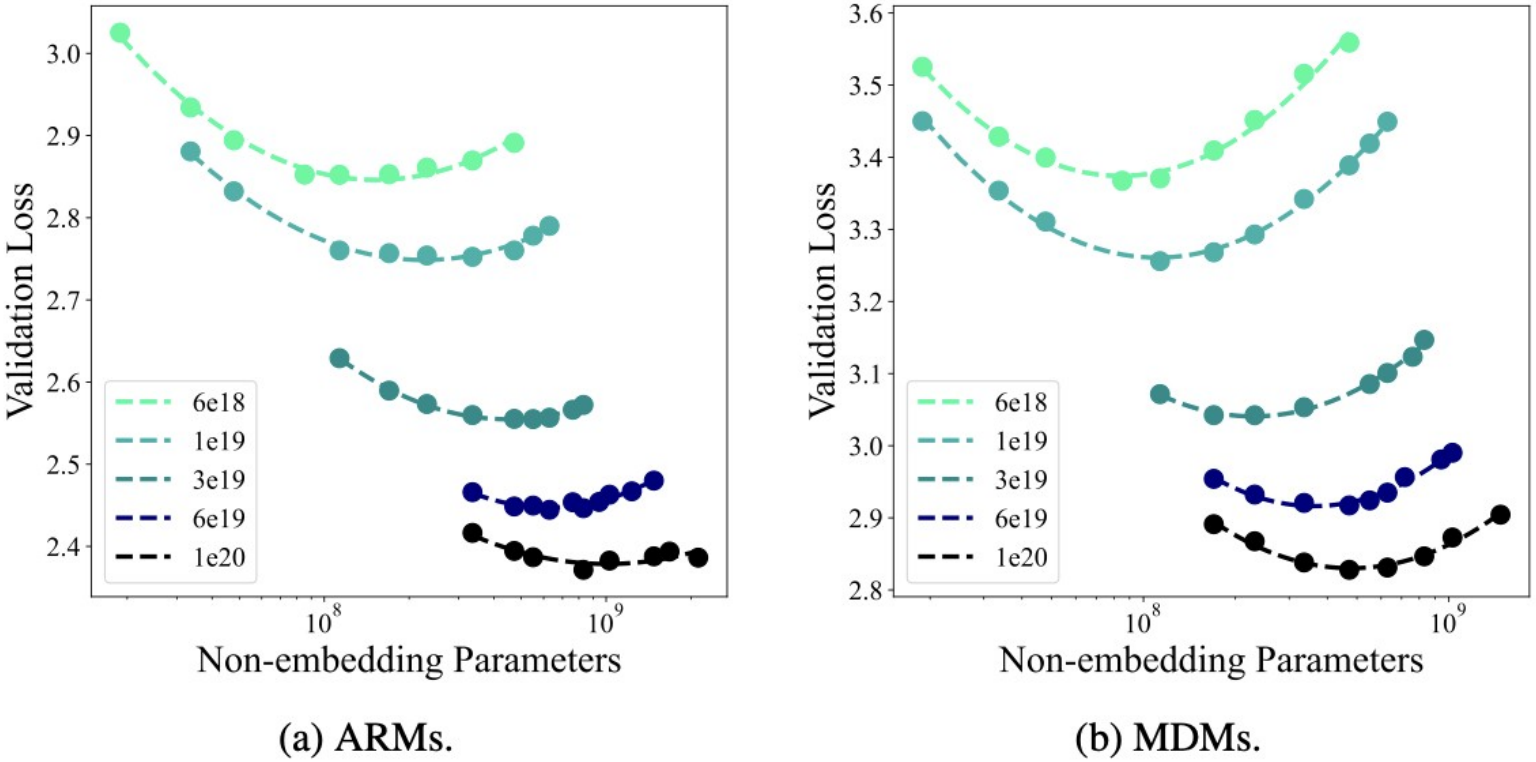
https://arxiv.org/pdf/2506.17298

Table 4: Co-Pilot Arena model comparison by latency, Elo scores, and ranks. Data obtained via Copilot Arena.

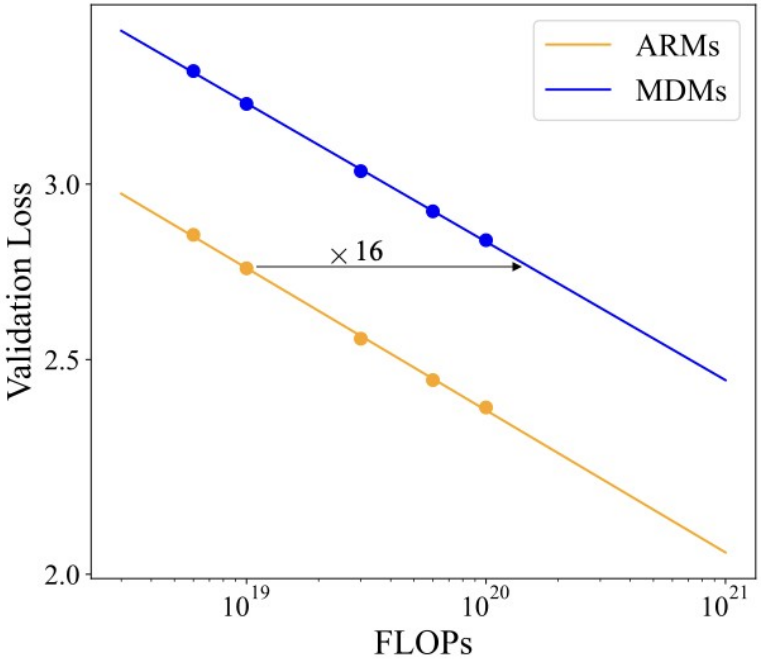
Model	Latency (seconds)	Latency Rank	Elo Score	Elo Rank
DeepSeek V2.5 (FIM)	2.07	11	1025	1
Claude 3.5 Sonnet	1.46	8	1003	1
Mercury Coder Mini	**0.25**	**1**	**993**	**2**
Codestral	0.31	2	992	2
Metal Llama 3.1 405B	1.84	10	982	3
GPT-4o	0.76	5	980	3
Gemini 1.5 Flash	0.59	3	977	3
Gemini 1.5 Pro	1.48	9	977	5
Meta Llama 3.1 70B	1.17	7	970	5
Qwen 2.5 Coder 32B	0.91	6	949	12
GPT-4o Mini	0.84	4	939	12

https://arxiv.org/pdf/2410.18514

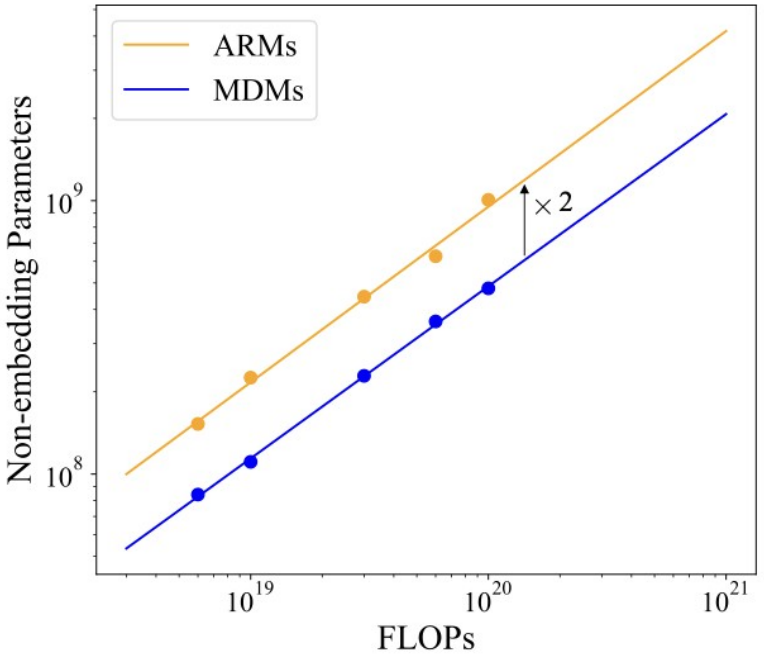
В чем подвох?



https://arxiv.org/pdf/2410.18514



(a) Loss-Flops curve.



(b) Parameters-Flops curve.